arXiv:2305.10425v1 [cs.CL] 17 May 2023

SLiC-HF: Sequence Likelihood Calibration with Human Feedback

Yao Zhao[†]
yaozhaoyz@google.com

Rishabh Joshi[†]
rishabhjoshi@google.com

Tianqi Liu[*]
tianqiliu@google.com

Misha Khalman[†]
khalman@google.com

Mohammad Saleh[†]
msaleh@google.com

Peter J. Liu[†]
peterjliu@google.com

Google Deepmind[†], Google Research[*]

Abstract

Learning from human feedback has been shown to be effective at aligning language models with human preferences. Past work has often relied on Reinforcement Learning from Human Feedback (RLHF), which optimizes the language model using reward scores assigned from a reward model trained on human preference data. In this work we show how the recently introduced Sequence Likelihood Calibration (SLiC), can also be used to effectively learn from human preferences (SLiC-HF). Furthermore, we demonstrate this can be done with human feedback data collected for a different model, similar to off-policy, offline RL data. Automatic and human evaluation experiments on the TL;DR summarization task show that SLiC-HF significantly improves supervised fine-tuning (SFT) baselines. Furthermore, SLiC-HF presents a competitive alternative to the PPO RLHF implementation used in past work while being much simpler to implement, easier to tune and more computationally efficient in practice.

1 Introduction

While massively scaling model parameters and training compute of Transformer-based language models have led to impressive few-shot in-context learning [5, 6], reinforcement learning from human feedback fine-tuning (RLHF) can significantly improve generation quality as judged by humans. This has been observed at all model scales for various downstream language generation tasks, such as abstractive summarization, dialogue, and creative writing [18, 3, 7, 13].

For summarization in particular, multiple studies have shown that summaries generated by models tuned with RLHF are preferred over the reference summaries in commonly used datasets [18, 10, 8]. Reference summaries are often mined from web documents and might not have the highest quality or preferred style. As a result, pure supervised learning, i.e. maximizing the likelihood of reference summaries given documents, is limited by the quality of reference summaries, thus additional feedback can improve models beyond the references. Commonly used reference-based metrics, such as ROUGE [11], only measure similarity between model generated and reference texts. These reference-based metrics cannot measure quality improvement beyond the reference summaries.

To implement RLHF, a reward model, $r_\phi(\mathbf{x}, \mathbf{y})$, is trained on human preference data, $(\mathbf{x}, \mathbf{y}_0, \mathbf{y}_1, i) \sim D_{HF}$, collected via side-by-side human evaluation, where raters are asked to judge which of the two summaries $\mathbf{y}_0$ and $\mathbf{y}_1$ is better for document $\mathbf{x}$, i.e. $i \in \{0, 1\}$. If we denote the preferred summary as $\mathbf{y}^+$ and the other $\mathbf{y}^-$, the human feedback becomes $(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-) \sim D_{HF}$. One common option

Preprint. Under review.

for the training loss of the reward model used by RLHF is:

$$loss(r_\phi) = -\mathbb{E}_{(\mathbf{x},\mathbf{y}^+,\mathbf{y}^-)\sim D_{HF}}[\log(\sigma(r_\phi(\mathbf{x},\mathbf{y}^+) - r_\phi(\mathbf{x},\mathbf{y}^-))] \tag{1}$$

Reinforcement learning algorithms such as PPO [17] are then used to refine a supervised fine-tuned model (SFT) to maximize the expected reward assigned by the reward model $r_\phi(\mathbf{x},\mathbf{y})$ [22, 18]. A KL-penalty term is typically added to the loss to prevent the RLHF model from diverging too far from the original supervised policy.

However, algorithms such as RLHF-PPO introduce significant complexity to the training process by adding separate value and reward networks that may be comparable in size to the policy network. They are typically kept in memory to maximize training speed, which for a given memory budget significantly reduces the maximum size of trainable model. Furthermore the optimization steps are significantly slower due to the use of roll-outs in the training loop, which involves sampling/decoding from the model. Hyper-parameter tuning and co-coordinating the PPO process is also more complex, requiring niche expertise.

Recently, another class of sequence-level contrastive methods [12, 21] seek to align model likelihood with an arbitrary, possibly non-differentiable reward, presenting an alternative to RL for optimizing the expected reward of samples. Zhao et al. [21] proposed Sequence Likelihood Calibration (SLiC) to align a language model's sequence likelihood, $p_\theta(y|x)$, over decoded sequences according to their similarity to reference sequences. The ranking *calibration* loss contrasts a positive sequence $\mathbf{y}^+$ and a negative sequence $\mathbf{y}^-$, encouraging the model P_θ to assign more probability mass to positive compared to negative sequences:

$$L^{\text{cal}}(\theta) = \max(0, \beta - \log P_\theta(\mathbf{y}^+|\mathbf{x}) + \log P_\theta(\mathbf{y}^-|\mathbf{x})) \tag{2}$$

While the original SLiC work used similarity to references as criteria for ranking, e.g. ROUGE [11] and model embedding distances, it can be replaced by an arbitrary, reference-less ranking function, $R(\mathbf{y}_0, \mathbf{y}_1, \mathbf{x}) \to \{0, 1\}$. Particularly in this work, we use human preference as the ranking function, either by using off-policy preference data D directly, or by training a predictive ranking model $R_\phi(\mathbf{y}_0, \mathbf{y}_1, \mathbf{x})$ from D.

We call using SLiC with this human preference ranking function SLiC-HF and apply it using the human feedback data collected in Stiennon et al. [18]. Our experiments show that SLiC-HF also leads to improved summarization quality on the Reddit TL;DR Summarization task as judged by humans, even though this feedback was collected for different models, similar to off-policy, offline RL. While our T5-Large [15] (770M parameter) SFT model performs similarly to Stiennon et al. [18]'s 6B decoder-only SFT model, we are able to improve our model with SLiC-HF such that it performs at least as well as Stiennon et al. [18]'s 6B RLHF-PPO model as judged by humans. Furthermore, applying SLiC-HF to the T5-XXL 11B parameter SFT model [15] significantly improves results.

The primary contributions of this paper are showing:

- how to apply SLiC to learn from human preferences (SLiC-HF), a simpler, more efficient yet competitive alternative to RLHF
- feedback/preference data from another model (off-policy) can be effectively leveraged by SLiC-HF, making it unnecessary to collect costly new feedback data for our model
- providing a general SLiC-HF recipe based on open-sourced T5 model that outperforms RLHF on the Reddit TL;DR summarization task

2 Method

In this work, we apply SLiC [21] to improve a SFT model using human preference data $(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-) \sim D_{HF}$ in addition to the standard supervised fine-tuning data $(\mathbf{x}, \mathbf{y}_{\text{ref}}) \sim D_{SFT}$.

2.1 Sequence Likelihood Calibration

Following Zhao et al. [21], we first fine-tune a supervised model, $P_{\theta_{ft}}(\mathbf{y}|\mathbf{x})$, on $(\mathbf{x}, \mathbf{y}_{\text{ref}}) \sim D_{SFT}$, and then align the SFT model's sequence likelihood using the SLiC approach which optimizes the following loss:

$$\mathcal{L}(\theta) = \sum L^{\text{cal}}(\theta, \mathbf{x}, \mathbf{y}_{\text{ref}}, \{\hat{\mathbf{y}}\}_m) + \lambda L^{\text{reg}}(\theta, \theta_{ft}; \mathbf{x}, \mathbf{y}_{\text{ref}}) \tag{3}$$

where θ and θ_{ft} are the current and fixed SFT model weights, L^{cal} and L^{reg} are the calibration and regularization losses and $\{\hat{\mathbf{y}}\}_m$ are m sampled candidates from the SFT model. More specifically, we choose the rank calibration loss and cross-entropy regularization loss for their simplicity and natural fit to pairwise human feedback data. Thus the loss function of SLiC-HF becomes the following:

$$\mathcal{L}(\theta) = max(0, \delta - \log P_\theta(\mathbf{y}^+|\mathbf{x}) + \log P_\theta(\mathbf{y}^-|\mathbf{x})) - \lambda \log P_\theta(\mathbf{y}_{\text{ref}}|\mathbf{x}) \quad (4)$$

The first term is the calibration loss where $\mathbf{x}$ is the input sequence, $\mathbf{y}^+$ and $\mathbf{y}^-$ are positive and negative sequences, and δ is a hyper-parameter for the margin of the ranking loss. The second term is the cross-entropy loss, where $\mathbf{y}_{\text{ref}}$ is some target sequence and λ is the regularization weight. Cross-entropy loss encourages the model to stay close to the SFT model, similar to the KL term in used in Stiennon et al. [18], however it does not need an extra copy of SFT weights. The KL regularization term was also explored in Zhao et al. [21] but found to perform similarly. The choices of $\mathbf{y}^+$ and $\mathbf{y}^-$ are discussed in subsections 2.2 and 2.3. The choices of regularization target $\mathbf{y}_{\text{ref}}$ is discussed in subsection 2.4.

2.2 SLiC-HF with Sample and Rank

Zhao et al. [21] samples candidates $\{\hat{\mathbf{y}}\}_m \sim P_{\theta_{ft}}(\mathbf{y}|\mathbf{x})$ from D_{SFT}'s training split, from which (positive, negative) pairs are determined. We call this approach *SLiC-HF-sample-rank*. To determine the rank, we consider two text-to-text models trained from the human preference data D_{HF}:

Trained Pointwise Reward model: Similar to Askell et al. [2], we binarize each ranked pair into a positive and a negative sequence, as shown in Figure 1. When training the reward model, input sequences are formatted as '[Context] ... [Summary] ... ' and target sequences are either 'Good' or 'Bad'. At inference time, we compute the probability of token 'Good' on the decoder side to score each of the m candidates in a list, and sample m positive/negative pairs from them.

Trained Pairwise Ranking model: As shown in Figure 1, we formulate the human feedback into a pairwise ranking problem with text-to-text format. When training the ranking model, input sequences are formatted as '[Context] ... [Summary A] ... [Summary B]' and target sequences are among 'A' or 'B'. At inference time, we use a tournament-style procedure to rank candidates in a list. For example, given a list of 4 candidates c_1, c_2, c_3, c_4, we first rank c_1, c_2 and c_3, c_4 and then rank $winner(c_1, c_2), winner(c_3, c_4)$. Given m candidates, the ranking model is called $m-1$ times and $m-1$ positive/negative pairs are yielded.

Reward Model
[CONTEXT] document [SUMMARY] positive summary → Good
[CONTEXT] document [SUMMARY] negative summary → Bad

Ranking Model
[CONTEXT] document [SUMMARY A] positive summary [SUMMARY B] negative summary → A
[CONTEXT] document [SUMMARY A] negative summary [SUMMARY B] positive summary → B

Figure 1: Training text-to-text reward model and ranking model.

2.3 SLiC-HF Directly On Human Feedback

We also consider a straight-forward approach of directly calibrating on positive and negative sequences from the human feedback dataset, D_{HF}, without a ranking or reward model. We call this approach *SLiC-HF-direct*. The obvious advantage of this approach is increased simplicity and efficiency from not training or using a ranking/reward model. SLiC-HF-direct does not incur additional engineering costs in decoding from the SFT model and training a model to label the decodes. The drawback is that the off-policy human feedback data distribution might differ much from the SFT model's decode distribution.

2.4 Regularization Term for Calibration

We consider two choices of target sequence $\mathbf{y}_{\text{ref}}$ for cross-entropy regularization. The first choice is using $\mathbf{y}_{\text{ref}}$ in D_{SFT} as regularization target. The second choice is using the best ranked candidate

from $\{\hat{\mathbf{y}}\}_m$ as the regularization target. Best ranked candidates can be selected using either the ranking model or the reward model.

3 Experimental Results

3.1 Datasets

We study SLiC-HF on Reddit TL;DR summarization datasets from Stiennon et al. [18]. The dataset contains both fine-tune data D_{SFT}, human feedback data D_{HF}, along with their SFT and RLHF model decodes which we use for comparison with our models. D_{SFT} is a filtered version of Reddit TL;DR dataset [19]. It contains 117k/6k/6k examples in train, validation and test splits. D_{HF} consists of 64k human preferences on decodes from multiple models.

3.2 Experimental Hyper-parameters

We conduct all experiments using T5 models [15] in the T5x framework [16]. In our ablation study, we choose a T5-large model (770M) as the generation model and T5-XXL (11B) as the ranking model and the reward model[1]. We train all generation models with batch size of 32 and ranking/reward models with batch size of 128. Both are trained with default learning rate of 10^{-3}.

We train the ranking model and the reward model on D_{HF} training split, and picked checkpoints that have the highest accuracy on D_{HF} validation split. We fine-tune T5 models on D_{SFT} training split, and pick checkpoints that have the lowest perplexity on D_{SFT} validation split.

In calibration, we use learning rate of 10^{-5} and ranking margin β of 1.0. When calibrating models on their own decodes with SLiC-HF-sample-rank, we sample 8 decodes with temperature of 0.7 and topk of 40 from fine-tuned only generation models.

When evaluating our models, we use beam-search with beam size 4. For automatic evaluation, we calculate the model decodes' win rate against human references measured by the T5-XXL ranking model on D_{SFT} validation dataset. Win rate is defined as the percentage of model decoded summaries preferred by the ranking model compared to human references.

3.3 Reward Model and Ranking Model Accuracy

Human feedback and human evaluation are done by raters comparing two summaries as it is more reliable than pointwise rating. We hypothesize that ranking model has an advantage over reward model because of its pairwise nature which aligns better with the task. We train and compare a T5-XXL ranking model and a T5-XXL reward model (subsection 2.2). Results shows that our ranking model has accuracy of 73.23% on D_{HF} validation, about 2% higher than our reward model which has accuracy of 71.34%[2].

3.4 SLiC Ablation

We conduct a set of experiments to ablate SLiC-HF settings against baselines. We use the ranking model as the main metric because of its higher correlation with human preferences demonstrated in Stiennon et al. [18]. Selected settings are later verified with our human evaluation experiments in subsection 3.5. We report ROUGE numbers just for reference purpose and do not use them to select models. It is expected to see a drop in ROUGE numbers when learning from human feedback because it has less incentive to be similar to the reference texts. Similar to RLHF in Stiennon et al. [18], we also observe an increase in average length of models and conduct a length controlled study in subsection 3.5.

3.4.1 SLiC-HF vs Continue Fine-tuning on Filtered Data

A simple way to learn from human feedback data is to convert it into SFT dataset and continue fine-tuning on it. In general, we use the filtering approach which has similar performance to controlled

[1] We find that smaller T5 ranking/reward models do not converge reliably in our setup.

[2] Our ranking and reward models' accuracy are similar to the 6B reward model in Stiennon et al. [18]

Table 1: Compare different methods to leverage human feedback data. Ranker win rate is the T5-XXL ranking model's preference of choosing model decodes over reference texts.

	Ablation			Metrics	
method	human feedback form	regularization	# words	R1 / R2 / RL	ranker win rate
reference		-	27.11	-	50%
SFT		-	23.57	35.1/12.87/26.81	44.96%
continue SFT on filtered data					
	positives from HF data	-	31.22	33.02/11.27/24.57	51.65%
	best decodes, by reward	-	27.69	35.31/12.41/26.21	63.24%
	best decodes, by ranking	-	28.26	35.39/12.69/26.56	65.43%
SLiC-HF					
	SLiC-HF-direct	SFT targets	41.03	33.76/11.58/24.72	82.92%
	SLiC-HF-sample-rank, by reward	SFT targets	38.44	33.87/11.48/24.81	82.42%
	SLiC-HF-sample-rank, by reward	best decodes	38.58	34.07/11.59/24.92	83.52%
	SLiC-HF-sample-rank, by ranking	SFT targets	37.96	34.49/11.92/25.35	**86.21%**
	SLiC-HF-sample-rank, by ranking	best decodes	37.50	34.69/12.03/25.54	**85.51%**

generation approaches [1] but is cleaner to implement. We consider three approaches to filter data for continued fine-tuning:

- keep only positive human feedback sequences and discard negative ones.
- decode 8 summaries from the SFT model, use the ranking model to select the best 1 out of 8 summaries by a tournament-style ranking approach.
- decode 8 summaries from the SFT model, use the reward model to select the best 1 out of 8 summaries by scoring each and taking the one with the max score.

As shown in Table 1, on Reddit TL;DR dataset, continue fine-tune on positive human feedback data improves model win rate against reference slightly from 44.96% to 51.65%. In this experiment, we choose to use all human feedback without filtering for better models because this mimics a real world scenario where we have access to some human feedback data without the explicit knowledge of its quality. Continuing fine-tuning on best 1 out of 8 further improves win rate against reference to 60%+ and using pairwise ranking model is slightly better than pointwise reward model for filtering.

3.4.2 Apply SLiC-HF Directly On Human Feedback Data

With SLiC-HF-direct, we observed that even though calibration loss decreases as expected, sequence length keeps increasing and does not converge to a stable value. On the other hand, SLiC-HF-sample-rank robustly converges. We hypothesize that SLiC-HF-direct is prune to out-of-distribution decodes generated by other models in the human feedback data.

When using the ranking model to select for the best checkpoint for SLiC-HF-direct, it has moderate length increment and has 82.92% win rate against reference which is close to SLiC-HF-sample-rank. The engineering complexity of SLiC-HF-direct is almost the same as fine-tuning a model. Therefore, it is a good candidate for quick experimentation on human feedback.

3.4.3 Apply SLiC-HF on Ranked Model Decodes

As shown in Table 1, SLiC-HF-sample-rank using the ranking model have about 3% gain in win rate against reference compared to SLiC-HF-sample-rank using the reward model. This results aligns with the observation in subsection 3.3 that the ranking model has higher agreement to human preference than the reward model.

For SLiC-HF-sample-rank using the ranking or the reward model, using SFT targets or best ranked decodes as regularization doesn't show much difference. This shows that SLiC-HF-sample-rank is applicable even when there is no ground truth reference available. The gain from continue fine-tuning on best ranked decodes in Table 1 is not additive to SLiC-HF.

3.5 Human Evaluation

We conduct side-by-side human evaluation between multiple systems using crowd-sourcing.[3] Given a document and 2-4 summaries, raters are tasked to assign a pointwise overall quality to each summary, select if the summary is factual or not, and choose the best summary.

Each task is replicated and judged by 3 different raters. To eliminate bias, we anonymize all the models and randomly shuffle order of summaries for each task. We aggregate pointwise metrics by averaging the ratings across all 3 crowd workers, and we aggregate the choice metric using majority vote.

The human evaluation template and the rating instructions can be found in Appendix A.

3.5.1 SLiC-HF Ablation Study

We conduct a 4-way side-by-side human evaluation to confirm the ablation results in Table 1. 100 examples from the validation set are sampled from reference, SFT model, continue fine-tuning model and SLiC-HF model (SLiC-HF-sample-rank, using ranking model, regularized on best decodes). As shown in Table 3, SLiC-HF is chosen as the best model 73% of the time, has significantly higher average quality, and is the most factual model. In general, the average quality aligns well with the ranker win-rate from Table 1.

Figure 2 shows the lengths controlled quality of SFT, continue fine-tuning and SLiC-HF models, which clearly shows SLiC-HF is preferred. Length controlled quality study is similar to studies conducted in Stiennon et al. [18], where mean scores are calculated among examples bucketed by their relative length to the reference.

Table 2: 4-way human evaluation to compare reference, SFT continue SFT on best decodes using ranking model, SLiC-HF with pairs of decodes using ranking model.

	reference	SFT	continue SFT	SLiC-HF	same
chosen as preferred %	13%	5%	5%	**73%**	4%
average quality	3.17	3.10	3.32	**3.82**	-
is factual %	94.16%	94.85%	94.85%	**96.56%**	-

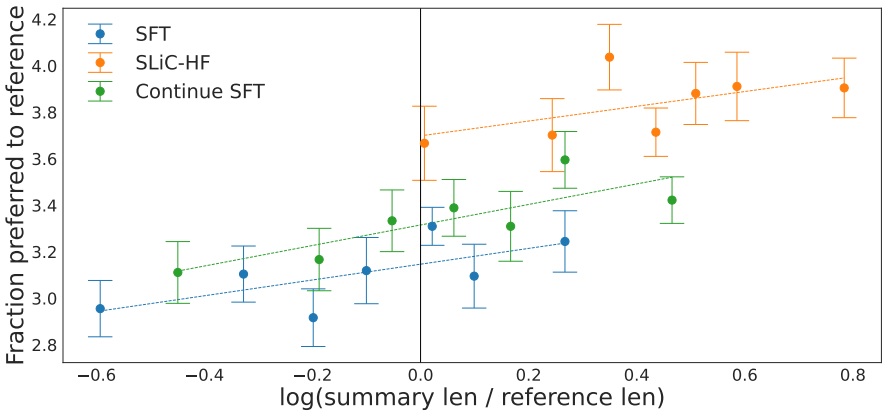

Figure 2: Length bucketed average quality of SFT and SLiC-HF against different baselines.

3.5.2 SLiC-HF vs RLHF-PPO

Correctly implementing and tuning the right hyper-parameters for the RLHF-PPO algorithms in Stiennon et al. [18] are non-trivial tasks. Instead of re-implementing the algorithms in our framework, we directly compare with the model decodes from Stiennon et al. [18].

[3] We use Amazon Mechanical Turk to set up the task and hire the raters

We first benchmark our T5-large SFT model against their 6B decoder-only SFT model in a two-way side-by-side human evaluation. As shown in Figure 3, our SFT has slightly higher quality and win rate but it is not statistically significant.

Next we benchmark two variants of our T5-large SLiC-HF-sample-rank models against the decoder-only 6B RLHF-PPO model from Stiennon et al. [18]. SLiC-HF-sample-rank with reward model has similar performance as RLHF-PPO and SLiC-HF-sample-rank with ranking model is better than the RLHF-PPO. The summaries from SLiC-HF models are slightly longer than the RLHF-PPO model, and their length controlled win rate is similar to RLHF-PPO as shown in Figure 3.

Table 3: Three 2-way side-by-side human evaluations to compare our SFT baseline with [18], and our SLiC-HF models with the RLHF-PPO model. Statistically significant results are denoted with *.

systems comparisons				human preference			
system A (ours)		system B([18])		win rate		quality	
method	# words		# words	A	B	A	B
SFT (770M gen)	23.7	SFT (sup6B)	24.6	56%	44%	3.59	3.48
SLiC-HF (700M gen, 11B ranking)	36.9	RLHF (sup6B_rm6B)	33.0	66%*	34%*	3.85*	3.61*
SLiC-HF (700M gen, 11B reward)	38.4	RLHF (sup6B_rm6B)	33.0	56%	44%	3.78	3.7

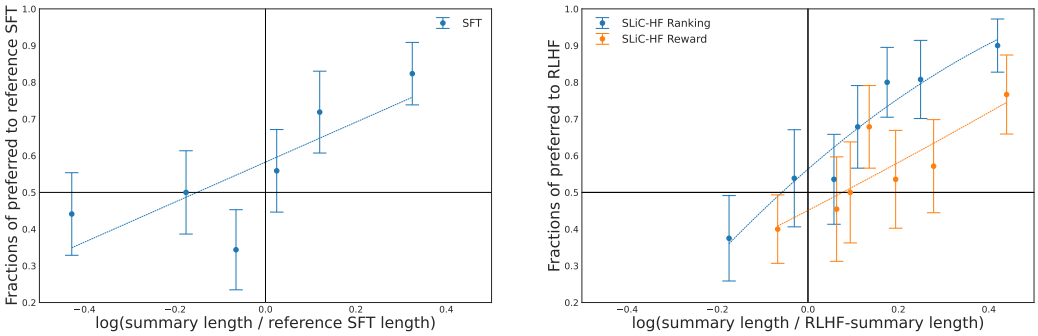

Figure 3: Length bucketed average quality of SFT and SLiC-HF against different baselines.

3.6 Scaling Up SLiC

Table 4: Effect of scaling up model parameters and number of candidates for SLiC-HF-sample-rank.

Ablation				Metrics	
method	# params	m	# words	R1 / R2 / RL	ranker win rate
SFT	770M	8	23.57	35.1/12.87/26.81	44.96%
SFT	11B	8	24.07	36.45/14.11/28.38	62.34%
SLiC-HF	770M	8	37.96	34.49/11.92/25.35	86.21%
SLiC-HF	770M	64	40.53	34.14/11.70/25.11	86.41%
SLiC-HF	11B	8	36.90	35.83/12.87/26.63	**96.10%**

We study 2 ways of scaling up the SLiC-HF-sample-rank: (1) scaling up generation model parameters, (2) scaling up number of decoded candidates m. As shown in Table 4, scaling up generation model from 770M to 11B significantly improves both the SFT model and the SLiC-HF model. On the other hand, scaling up m from 8 to 64 does not help much.

4 Further discussion on SLiC-HF vs. RLHF-PPO

4.1 Compute/Memory Efficiency and Parallelism

We summarize the compute and memory efficiency differences between SLiC-HF and RLHF-PPO in Table 5.

Table 5: Compute and memory efficiency comparison. p denotes the number of parameters in the policy network;

	RLHF-PPO [18]	SLiC-HF	
		decode-rank	direct
Auxiliary models	reward, value, SFT	ranking	-
Decoded sequences	1M	800k	-
Parameter memory usage for training	$4p$	p	p
Parameter updates per step	$2p$	p	p
Parallel decoding	within batch	whole training set	-
Parallel reward	within batch	whole training set	-
Input encoding caching	no	yes	-

In both RLHF-PPO and SLiC-HF-sample-rank we train an auxiliary ranking or reward model that is used to judge the quality of summaries. However, Stiennon et al. [18] found that having separate policy and value networks worked significantly better, and thus contributes an extra auxiliary model, the same size as the reward model that is updated along-side policy updates.

Furthermore, the policy, value, reward, and SFT models (all the same size in Stiennon et al. [18]) are used within the PPO training loop. They are often held in hardware memory to ensure faster training steps. Whereas in SLiC-HF, the rewards can be computed completely in parallel and offline, thus using 1/4 the memory for model weights during training. Such memory savings could be re-purposed to train larger models.

Stiennon et al. [18] report using 1M episodes to conduct RLHF training, which corresponds to roughly the same number of decoded samples used in SLiC-HF, ($m = 8$ per training example, 123,169 examples). However, in practice SLiC-HF decoding can be significantly faster because all the decoded samples use the same policy allowing for completely parallel decoding. In contrast, with PPO the policy is updated every batch, limiting decoding parallelism to each batch (512, in [18]) as subsequent decoding is blocked on policy updates. Furthermore, PPO decoding occurs within the training loop leading to much longer optimization step times. Whereas with SLiC-HF step times are similar to fine-tuning, which is significantly faster as there is no decoding in the training loop.

Beyond the significant decoding parallelism gains, SLiC-HF can make use of simple input encoding caching optimizations to reduce compute. Since the m decodes are sampled from the same SFT policy, the input sequence encoded states can be cached rather than recomputed. In summarization and other tasks involving long contexts, this may be significant as input sequence length tends to be much longer than output.

SLiC-HF has similar parallelism advantages in computing rewards per episode compared to RLHF as the ranking can be computed outside the training loop instead of within.

4.2 Pairwise Ranking vs Reward model

RL algorithms seek to maximize the expected reward of trajectories, in this case the human judgement in quality of model summaries. This reward function typically is assumed to be pointwise, whereas human preference data is collected pairwise to improve reliability. Thus there is noise introduced in converting pairwise judgements into pointwise rewards, which can be estimated as the difference in ranking accuracy as in subsection 3.3. Since SLiC-HF only cares about the relative rank of two summaries, this pairwise-to-pointwise noise is avoided and we conjecture this helps SLiC-HF (Table 1, Figure 3).

4.3 The Value of States and Actions in Language

For many tasks tackled using RL, rewards may be collected at the end of a trajectory (as in many Atari games) and the attribution of final reward to specific actions may be very important in learning to solve a task. Typically when RL is applied to language as in the RLHF literature, the state is the prefix of the current text and the actions correspond to choosing the next token. The value function's role is to estimate the goodness of a trajectory (e.g. summary) from a prefix/input, which is intuitively a very difficult task for human raters, and thus RL may also suffer from value function estimation

noise. In contrast, SLiC-HF does not rely on such a sub-model and only uses the cleaner preference signal to drive parameter updates and leading to what we conjecture is more stable optimization.

5 Related work

RL has been used to optimize arbitrary reward in language generation such as BLEU for translation [20] and ROUGE for summarization [14]; however, while those metrics improved, human judgement of quality suffered due to metrics misalignment.

In an effort to better align the reward function with human judgement, many works used RL to align language models with a reward model trained to predict carefully collected human judgements [22, 18, 13] using summarization as an initial proof-of-concept. A KL penalty term, first used in Jaques et al. [9], is used as regularization to prevent the tuned model from departing from the initial supervised model, and is also used in SLiC [21].

Liu et al. [12] propose BRIO, which has a similar intent as SLiC [21] of rank-ordering model-generated decodes according to a reward function. BRIO trains models to align length normalized sequence probability of generated decodes to their similarity to reference as measured by ROUGE using a list-wise loss function. In contrast, and similar to RLHF, SLiC-HF adapts the technique to align with a model trained to predict human preference given two summaries instead of their similarity to the reference.

Bai et al. [4] substitutes human preference data with judgements from a large language model, and calls it AI feedback (AIF). SLIC-HF can also be used with AIF exactly in the same way and is indifferent about the AI or human origin of the feedback.

6 Conclusion

In this work, we proposed SLiC-HF that calibrates sequence likelihood on human feedback data. Our experiments on the Reddit TL;DR summarization task show that SLiC-HF significantly improves supervised fine-tuning (SFT) baselines, and presents a competitive alternative to the RLHF-PPO implementation of past work while being simpler to implement, easier to tune and computationally efficient. Future work may include studying SLiC-HF on other language generation tasks using other reward functions and/or non-human feedback.

References

[1] Roee Aharoni, Shashi Narayan, Joshua Maynez, Jonathan Herzig, Elizabeth Clark, and Mirella Lapata. 2022. mface: Multilingual summarization with factual consistency evaluation.

[2] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. 2021. A general language assistant as a laboratory for alignment.

[3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.

[4] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

[6] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

[7] Amelia Glaese, Nat McAleese, Maja Trębacz, John Aslanides, Vlad Firoiu, Timo Ewalds, Maribeth Rauh, Laura Weidinger, Martin Chadwick, Phoebe Thacker, et al. 2022. Improving alignment of dialogue agents via targeted human judgements. *arXiv preprint arXiv:2209.14375*.

[8] Tanya Goyal, Junyi Jessy Li, and Greg Durrett. 2022. News summarization and evaluation in the era of gpt-3. *arXiv preprint arXiv:2209.12356*.

[9] Natasha Jaques, Shixiang Gu, Dzmitry Bahdanau, José Miguel Hernández-Lobato, Richard E Turner, and Douglas Eck. 2017. Sequence tutor: Conservative fine-tuning of sequence generation models with kl-control. In *International Conference on Machine Learning*, pages 1645–1654. PMLR.

[10] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*.

[11] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81.

[12] Yixin Liu, Pengfei Liu, Dragomir Radev, and Graham Neubig. 2022. BRIO: Bringing order to abstractive summarization. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2890–2903, Dublin, Ireland. Association for Computational Linguistics.

[13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

[14] Romain Paulus, Caiming Xiong, and Richard Socher. 2017. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*.

[15] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

[16] Adam Roberts, Hyung Won Chung, Anselm Levskaya, Gaurav Mishra, James Bradbury, Daniel Andor, Sharan Narang, Brian Lester, Colin Gaffney, Afroz Mohiuddin, Curtis Hawthorne, Aitor Lewkowycz, Alex Salcianu, Marc van Zee, Jacob Austin, Sebastian Goodman, Livio Baldini Soares, Haitang Hu, Sasha Tsvyashchenko, Aakanksha Chowdhery, Jasmijn Bastings, Jannis Bulian, Xavier Garcia, Jianmo Ni, Andrew Chen, Kathleen Kenealy, Jonathan H. Clark, Stephan Lee, Dan Garrette, James Lee-Thorp, Colin Raffel, Noam Shazeer, Marvin Ritter, Maarten Bosma, Alexandre Passos, Jeremy Maitin-Shepard, Noah Fiedel, Mark Omernick, Brennan Saeta, Ryan Sepassi, Alexander Spiridonov, Joshua Newlan, and Andrea Gesmundo. 2022. Scaling up models and data with `t5x` and `seqio`. *arXiv preprint arXiv:2203.17189*.

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

[18] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. 2020. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021. Curran Associates, Inc.

[19] Michael Völske, Martin Potthast, Shahbaz Syed, and Benno Stein. 2017. TL;DR: Mining Reddit to learn automatic summarization. In *Proceedings of the Workshop on New Frontiers in Summarization*, pages 59–63, Copenhagen, Denmark. Association for Computational Linguistics.

[20] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

[21] Yao Zhao, Mikhail Khalman, Rishabh Joshi, Shashi Narayan, Mohammad Saleh, and Peter J Liu. 2023. Calibrating sequence likelihood improves conditional language generation. In *The Eleventh International Conference on Learning Representations*.

[22] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2020. Fine-tuning language models from human preferences.

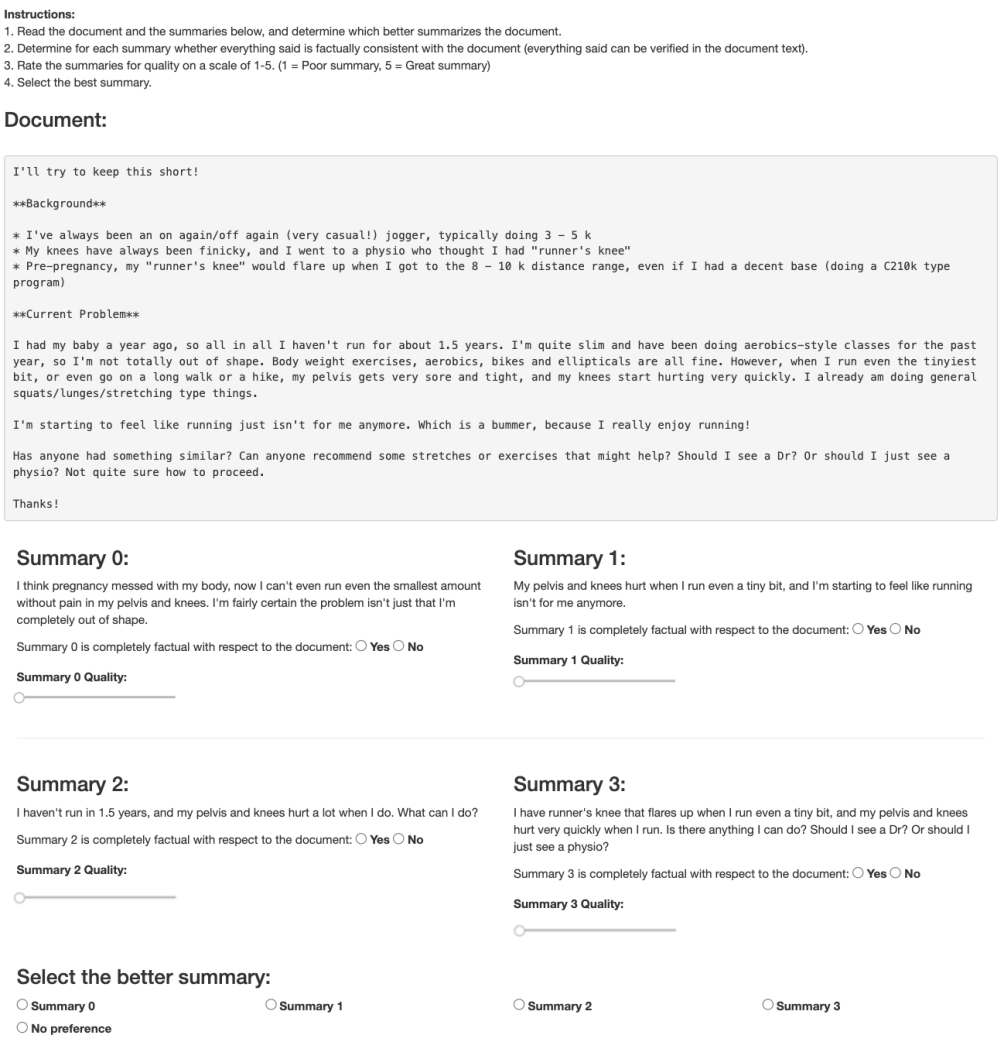

Figure 4: Example of human evaluation task.

A Human Evaluation

See Figure 4 for an example of the human evaluation task with 4 summaries. Summaries are randomly shuffled for each example and models are anonymized.